



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en Informática

Asignatura: Ingeniería del Software II

Unidad 5: Gestión de Configuración, Versionamiento y DevOps

Integrantes	C.i
Arnaldo Perdomo	30.791.551
Daniel Reyna	29.989.066
Jhoanny Maita	30.694.732
Yoryelis Ocando	31.074.651

Puerto Ordaz, 22 de Septiembre del 2026

Unidad 5: Gestión de Configuración, Versionamiento y DevOps

1. Pregunta

Un equipo que desarrolla un SaaS de facturación bajo un modelo GitHub Flow. En un mismo momento concurren dos eventos: un desarrollador finaliza la funcionalidad de exportación a PDF en su rama `feature/pdf-export`, y se detecta un error crítico en producción que impide el inicio de sesión de los usuarios sobre la rama `main`. El objetivo fue diseñar el flujo operativo completo comandos Git y acciones en plataforma para publicar la funcionalidad de forma segura y, al mismo tiempo, resolver el hotfix de producción sin contaminar el trabajo en curso. Adicionalmente, se diseñó la plantilla de Pull Request para la funcionalidad y se definieron los Quality Gates que el pipeline de CI debe validar antes de habilitar el merge.

Se organizó un repositorio con cuatro artefactos principales:

- `WORKFLOW.md`: documentación formal del flujo, con diagrama Mermaid y secuencia de comandos Git para `feature` y `hotfix`.
- `.github/pull_request_template.md`: plantilla estandarizada de Pull Request adaptada al módulo de exportación a PDF.
- `.github/workflows/ci.yml`: pipeline de Integración Continua con Quality Gates alineados a DevSecOps.
- `README.md`: presentación del repositorio y su estructura.

El flujo se resolvió en cuatro fases, priorizando siempre la estabilidad de producción:

- **Fase I — Aislamiento por emergencia.** Ante el fallo crítico, el trabajo en curso de la `feature` se preservó con `git stash -u`, evitando perder cambios o contaminar el repositorio. Luego se sincronizó `main` y se creó la rama efímera `hotfix/login-auth-failure`.
- **Fase II — Resolución y despliegue del hotfix.** Se implementó la corrección, se publicó la rama y se abrió un Pull Request urgente hacia `main`. El pipeline ejecutó validaciones rápidas y, tras aprobación del Tech Lead, se realizó el merge y se etiquetó la versión `v1.1.1`.
- **Fase III — Reanudación de la feature.** Se retomó `feature/pdf-export`, se recuperó el estado con `git stash pop` y se reintegró el nuevo estado de `main` mediante `git rebase main`, garantizando compatibilidad con el hotfix.
- **Fase IV — Publicación de la funcionalidad.** Se subió la rama con `git push --force-with-lease` (opción segura tras un rebase) y se abrió el Pull Request hacia `main`, donde el pipeline completo de Quality Gates y la revisión por pares determinaron el merge.

La plantilla incluye secciones que cubren lo solicitado por el PDF y refuerzan la trazabilidad:

- **Issue Relacionado:** vincula el cambio a un ticket.
- **Descripción del Cambio:** tipo de cambio y componente afectado (Módulo de Exportación a PDF).
- **Impacto Arquitectónico y de Negocio:** efectos en el sistema y rendimiento.
- **Evidencia:** capturas, logs y resultados de pruebas.

- **Plan de Rollback:** pasos concretos de reversión.
- **Checklist de Calidad y Seguridad:** incluye explícitamente las preguntas solicitadas, pruebas unitarias con cobertura mínima del 80% y auditoría de seguridad de la librería PDF contra CVEs, además de verificación de secretos, gestión de memoria/concurrencia y análisis estático.

Se definieron **tres Quality Gates** que el pipeline valida automáticamente antes de habilitar el merge:

1. **Gate 1 — Calidad de Código:** linting, análisis estático de seguridad (SAST con Semgrep) y verificación de compilación (npm run build).
2. **Gate 2 — Pruebas y Cobertura:** ejecución de pruebas unitarias con umbral mínimo del 80% en branches, functions, lines y statements.
3. **Gate 3 — Seguridad:** auditoría de dependencias (npm audit --audit-level=high) y escaneo de fuga de secretos con Gitleaks.

Complementariamente, en WORKFLOW.md se documentó la **configuración de Branch Protection** sobre main: exigir Pull Request, mínimo una aprobación, descartar aprobaciones obsoletas, requerir status checks, prohibir push directo y exigir historial lineal. Esta configuración es la que realmente convierte los gates en un bloqueo efectivo del botón de merge.

Decisiones técnicas relevantes

- Se priorizó el **hotfix** sobre la feature por el impacto en producción.
- Se usó git stash -u para no perder trabajo ni generar commits temporales.
- Se aplicó git rebase en lugar de merge para mantener historial lineal.
- Se usó --force-with-lease para evitar sobrescribir trabajo ajeno tras el rebase.
- Se etiquetó el hotfix con versionado semántico (v1.1.1) para trazabilidad.
- Se incorporó SAST y secret scanning para alinear el pipeline a DevSecOps (Shift-Left Security), tal como sugiere la unidad.

2. Pregunta

Análisis de Modelos de Ramificación: Compare rigurosamente los modelos GitFlow y Trunk-Based Development. Utilizando los conceptos del libro Accelerate (Forsgren et al.), argumente por qué la industria de alto rendimiento se está moviendo hacia Trunk-Based. ¿Qué prácticas de SCM ejemplo CI automatizado, son obligatorias para que Trunk-Based no resulte en un código inestable en producción?

A continuación, se presenta un Dockerfile básico y comentado para una aplicación Node.js cuyo archivo principal es server.js y que expone el servicio en el puerto 8080:

```
# Imagen base oficial de Node.js en su variante Alpine ligera y minimalista
FROM node:20-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos primero los manifiestos de dependencias
# aprovecha el cacheo de capas de Docker si no cambian,
```

```
# npm install no se vuelve a ejecutar en builds posteriores
COPY package*.json ./

# Instalamos únicamente las dependencias de producción
RUN npm install --omit=dev

# Copiamos el resto del código fuente de la aplicación
COPY . .

# Documentamos el puerto en el que escucha la aplicación
EXPOSE 8080

# Buena práctica de seguridad, evitar ejecutar el proceso como root
USER node

# Comando de arranque del contenedor
CMD ["node", "server.js"]
```

La deriva de configuración configuration drift ocurre cuando los entornos de desarrollo, prueba y producción divergen silenciosamente porque cada uno fue configurado de forma manual, en momentos distintos y por personas distintas. El Dockerfile ataca directamente este problema porque convierte la definición del entorno de ejecución en un artefacto de texto, versionado y determinista:

Fija de manera explícita la versión de la imagen base (node:20-alpine), evitando que la última versión disponible cambie de un servidor a otro.

Usa package*.json y su lockfile para instalar exactamente las mismas versiones de dependencias en cualquier máquina donde se construya la imagen.

Empaqueta la aplicación junto con su entorno de ejecución en una imagen inmutable, una vez construida, la imagen no se modifica in situ, si se necesita un cambio, se construye una nueva imagen y se reemplaza el contenedor.

Al vivir el Dockerfile dentro del repositorio Git, cualquier desarrollador, el pipeline de CI o el equipo de operaciones parte exactamente de la misma definición, eliminando el histórico funciona en mi máquina.

Por lo que el Dockerfile traslada la responsabilidad de la configuración del entorno de procedimientos manuales y propensos a errores, a una especificación declarativa y reproducible que puede construirse de forma idéntica en desarrollo, pruebas y producción.

Reflexión de la trazabilidad y auditoría de IaC en Git frente a los logs manuales

Es evidente que tratar la infraestructura como código y gestionarla en Git ofrece beneficios de trazabilidad y auditoría que los registros manuales de un administrador de sistemas simplemente no pueden igualar.

Historial completo y verificable: cada cambio queda registrado como un commit con autor, fecha exacta y un diff preciso de qué línea cambió, algo que un log manual rara vez documenta con ese nivel de detalle ni de forma consistente.

Revisión por pares antes de aplicar el cambio: mediante Pull Requests, cualquier modificación a la infraestructura pasa por una revisión de código antes de fusionarse, algo que sustituye de forma más ágil y sistemática a la antigua Junta de Control de Cambios CCB.

- Reversión trivial: si una configuración introduce un problema, git revert regresa el entorno a un estado exactamente conocido y probado con anterioridad; deshacer un cambio manual documentado en un log requiere reconstruir el paso a paso de memoria.
- Reproducibilidad determinista: el mismo commit siempre genera la misma infraestructura, mientras que un procedimiento manual está sujeto a la interpretación y a errores humanos de quien lo ejecute.
- Auditoría de cumplimiento: comandos como git log o git blame permiten responder en segundos preguntas como quién cambió esto, cuándo y por qué, algo que un log de sistema disperso y no estructurado dificulta enormemente en una auditoría real.

Se puede decir que Git no solo almacena el código de la infraestructura, al aplicar sobre ella los mismos principios clásicos de la Gestión de Configuración de Software el cual es identificación, control de versiones, control de cambios y auditoría, convierte cada cambio de entorno en un evento íntegro, trazable y reproducible de manera determinista.

GitFlow vs Trunk-Based Development

GitFlow y Trunk-Based Development representan dos filosofías opuestas frente a la integración del código. GitFlow, propuesto por Vincent Driessen (2010), se apoya en ramas de larga vida main y develop complementadas por ramas temporales de feature, release y hotfix. Este modelo ofrece un control muy estructurado ideal para software con ciclos de liberación largos y versiones que deben mantenerse en paralelo pero como contrapartida introduce integraciones tardías del código y una alta probabilidad de conflictos de fusión acumulados los llamados merge hells.

Trunk-Based Development en cambio, propone que todo el equipo integre sus cambios directamente sobre una única rama compartida (trunk o main), varias veces al día, o mediante ramas de vida extremadamente corta. Para ocultar funcionalidad incompleta sin bloquear la integración continua, TBD se apoya en Feature Flags que permiten desplegar código a producción en estado latente y activarlo cuando esté listo.

¿Por qué la industria de alto rendimiento se mueve hacia Trunk Based?

El libro Accelerate (Forsgren, Humble y Kim, 2018) respalda esta tendencia con evidencia empírica derivada del programa de investigación DORA DevOps Research and Assessment. Los autores identificaron que desarrollar sobre trunk/master en lugar de sobre ramas de características de larga vida, está correlacionado con un mayor desempeño de entrega de software. Los equipos de alto desempeño mantenían menos de tres ramas activas en cualquier momento, sus ramas vivían menos de un día antes de fusionarse al trunk, y nunca aplicaban períodos de code freeze o estabilización, los autores subrayan que este hallazgo es independiente del tamaño del equipo, de la organización o de la industria.

Este desempeño se mide según el marco de Accelerate a través de cuatro métricas las cuales son el lead time de entrega que es el tiempo desde que se confirma un cambio de código hasta que corre exitosamente en producción, la frecuencia de despliegue, el tiempo de restauración del servicio ante una falla, y la tasa de fallos en los cambios change fail rate. Trunk Based Development al reducir drásticamente el tamaño de cada lote de cambios y forzar la integración constante,

favorece directamente las dos primeras métricas lead time corto y alta frecuencia de despliegue sin sacrificar estabilidad cuando se combina con las prácticas de soporte adecuadas.

Prácticas obligatorias para que TBD no genere inestabilidad en producción

Integrar directamente en una rama compartida varias veces al día solo es seguro si se acompaña de un conjunto de prácticas de gestión de código fuente SCM y de ingeniería que actúan como una red de seguridad.

Primero la Integración Continua (CI) debe ser automatizada y obligatoria, cada vez que se integre al trunk debe activarse de inmediato un pipeline que compile el código, ejecute pruebas automatizadas y realice un análisis estático, por lo que si algún Quality Gate falla, la fusión debe ser bloqueada.

Además, es crucial contar con una suite de pruebas automatizadas que sea robusta y rápida. Sin una cobertura confiable de pruebas unitarias de integración y si es posible de contrato, la integración constante en una sola rama solo acelerará la llegada de errores a producción.

Los Feature Flags son otra herramienta valiosa ya que permiten desacoplar el despliegue del código de la liberación de la funcionalidad, esto significa que el trabajo incompleto puede integrarse al trunk sin que los usuarios finales lo vean.

Si se utilizan ramas de características estas deben tener una vida ultra corta fusionándose en menos de un día para evitar divergencias prolongadas que compliquen la integración.

El feedback rápido del pipeline es esencial, un pipeline de CI que tarda demasiado desincentiva la integración frecuente, el tiempo de retroalimentación debe ser de minutos y no de horas.

Por último, los despliegues progresivos de bajo riesgo como los canary releases y rolling updates, permiten validar en producción con una exposición controlada antes de liberar el cambio al 100% de los usuarios.

3. Pregunta

Evolución del Control de Cambios): "La Junta de Control de Cambios (CCB) tradicional es un cuello de botella que viola los principios de la Entrega Continua". Discuta esta afirmación. Explique cómo un Pull Request combinado con un Pipeline de Integración Continua (CI) implementa un "Control de Cambios Automatizado" que es más seguro, rápido y auditable que el proceso manual.

Por qué la Junta de Control de Cambios es un cuello de botella

La Junta de Control de Cambios es un comité de personas que se reúne cada cierto tiempo para aprobar o rechazar cada modificación. En la Entrega Continua el software debe estar siempre listo para desplegarse, y los equipos integran cambios varias veces al día. Si cada cambio tiene que esperar la reunión de un comité, se acumulan cambios pendientes y las entregas se hacen más grandes, y por tanto más riesgosas. Además, los miembros de la junta suelen estar lejos del código. Aprueban con base en documentos y resúmenes, no en evidencia técnica real.

Cómo lo resuelve el Pull Request con Integración Continua

El control de cambios no desaparece, se traslada al flujo de trabajo. Funciona en dos capas:

1. **El Pull Request** es la revisión por pares. Otro desarrollador, que sí conoce el código, revisa la lógica, el impacto en la arquitectura y la seguridad. La revisión ocurre en horas, no en semanas.
2. **El pipeline de Integración Continua** actúa como un juez automático e imparcial. Al abrir el Pull Request, es decir, una solicitud para incorporar un cambio al código principal, se compila el código, se ejecutan las pruebas, se hace el análisis estático y se revisan las dependencias en busca de vulnerabilidades. Si no se cumplen las puertas de calidad, el botón de fusionar queda bloqueado.

Por qué es más seguro, más rápido y más auditable

- **Más seguro:** la junta decide con opiniones sobre documentos; el pipeline decide con pruebas ejecutadas sobre el código real. Además, como los cambios son pequeños y frecuentes, cada uno es fácil de revisar y, si falla, fácil de revertir.
- **Más rápido:** la validación tarda minutos y corre todos los días, a cualquier hora, sin esperar una reunión. Se puede aprobar y desplegar varias veces al día sin sacrificar control.
- **Más auditable:** en el proceso manual, la evidencia son actas y firmas que pueden perderse o incompletarse. Con Pull Request e Integración Continua, todo queda registrado automáticamente en el repositorio: quién propuso el cambio, quién lo revisó, qué pruebas pasaron y cuándo se fusionó. Ese historial no se puede alterar sin dejar rastro, y sirve de auditoría permanente.

4. Pregunta

Artefactos y Auditoría: En la SCM clásica, el "Plan de Gestión de Configuraciones" era un documento de Word. En la era DevOps, este plan se materializa en archivos de texto dentro del repositorio. Identifique y explique la función de al menos 4 archivos de configuración tales como `.gitignore`, `Dockerfile`, `.github/workflows/ci.yml` y cómo cada uno garantiza la integridad, reproducibilidad y control de un ítem de configuración moderno.

En la Gestión de Configuración de Software (SCM) clásica, el "Plan de Gestión de Configuraciones" era un documento estático, típicamente redactado en Word, que describía de forma manual cómo se identificarían, controlarían y auditarían los ítems de configuración (CI) del proyecto: código, documentación, compiladores, entornos. Su principal debilidad era que ese plan vivía separado del propio código: alguien debía mantenerlo actualizado a mano, y su historial de cambios si existía dependía de la disciplina humana, no de una herramienta que lo garantizara.

En la era DevOps, ese mismo plan se materializa como archivos de texto plano dentro del repositorio, versionados por Git igual que el código fuente. Esto significa que cualquier modificación a la configuración del proyecto queda sujeta al mismo proceso de control de cambios (commits, revisiones por pares, historial auditable) que el código de la aplicación. A continuación, se identifican y explican cuatro de estos archivos modernos, y cómo cada uno garantiza integridad, reproducibilidad y control de un ítem de configuración.

1. gitignore

- **Función:** define qué archivos o carpetas no deben incorporarse al control de versiones (por ejemplo, `node_modules/`, archivos `.env` con credenciales, binarios compilados o logs generados localmente).

- **Garantía de integridad y control:** evita que artefactos generados de forma local que varían de una máquina a otra contaminen el repositorio compartido. Sin este archivo, cada desarrollador podría subir accidentalmente dependencias, binarios o secretos distintos, rompiendo la reproducibilidad del entorno y exponiendo información sensible. El .gitignore actúa, así como una primera línea de defensa que preserva la limpieza y la consistencia del ítem de configuración "repositorio".

2. Dockerfile

- **Función:** especifica, de forma declarativa y paso a paso, cómo construir la imagen del entorno de ejecución de la aplicación: imagen base, dependencias del sistema, variables de entorno, puerto expuesto y comando de arranque.
- **Garantía de integridad y control:** constituye la definición formal y versionada del entorno de ejecución. Mientras el Dockerfile no cambie, la imagen resultante de construirlo será, en esencia, siempre la misma, sin importar quién la construya ni en qué máquina lo haga. Esto materializa el concepto clásico de línea base: un estado revisado y acordado que solo puede modificarse mediante un commit explícito y trazable, eliminando la deriva de configuración entre entornos de desarrollo, pruebas y producción.

3. github/workflows/ci.yml (o equivalente en GitLab CI / Jenkinsfile)

- **Función:** define el pipeline de Integración Continua: qué pruebas ejecutar, qué análisis estático aplicar (SAST, linters) y bajo qué condiciones (Quality Gates) se autoriza la fusión de un cambio.
- **Garantía de integridad y control:** convierte el control de calidad en código auditable. Ya no depende de que una persona recuerde ejecutar las pruebas antes de aprobar un cambio; el propio archivo versionado impone las reglas de forma consistente para todo el equipo. Su historial en Git muestra, además, exactamente cuándo y por qué cambió el criterio de validación, quedando registrado el mismo tipo de trazabilidad que antes exigía un comité de control de cambios.

4. Archivos de Infraestructura como Código (por ejemplo, main.tf de Terraform o un manifiesto YAML de Kubernetes)

- **Función:** describen de forma declarativa el estado deseado de la infraestructura: servidores, redes, políticas de seguridad y demás recursos de cómputo.
- **Garantía de integridad y control:** elimina la deriva de configuración entre entornos, ya que cualquier cambio manual realizado directamente sobre un servidor, sin pasar por el archivo versionado, queda expuesto como una divergencia que herramientas como Terraform o agentes GitOps pueden detectar y revertir automáticamente. De esta manera, todo cambio real a la infraestructura queda trazado en el historial de Git, con autor, fecha y justificación.

Estos cuatro archivos. gitignore, Dockerfile, el archivo de pipeline de CI y los manifiestos de Infraestructura como Código cumplen exactamente las mismas funciones clásicas de la SCM: identificación de ítems de configuración, establecimiento de líneas base, control de cambios y auditoría de configuración. La diferencia radica en que hoy estas funciones se ejecutan de forma automatizada, versionada y ejecutable, en lugar de depender de un proceso burocrático y documental. En este sentido, cada git log de estos archivos es, en sí mismo, el registro de auditoría que antes requería la intervención manual de un comité de control de cambios, logrando una gestión de configuraciones más rápida, consistente y verificable.

